Indicaciones generales. La evidencia consiste en un programa en Python que resuelva un problema de optimización relacionado con la distribución de última milla mediante un algoritmo bioinspirado estudiado en la unidad de formación (UF). Se entregarán código fuente, informe técnico y reflexión individual; además, se realizará una evaluación presencial con preguntas individuales. **La evaluación y retroalimentación de las evidencias se realiza en Canvas.**

Subcompetencias asociadas

Simulación computacional	**SMA0103**
Reconocimiento de patrones, lenguaje natural e IA	**SMA0402**

1. Selección del problema y algoritmo bioinspirado

Seleccione uno de los problemas de optimización de última milla presentados durante la introducción a la situación problema. Identifique las decisiones a optimizar y sus restricciones. Seleccione y justifique un algoritmo estudiado en clase, como algoritmos genéticos (GA), colonia de hormigas (ACO), enjambre de partículas (PSO) o algoritmo de abejas (BA).

2. Definición y caracterización del problema

Describa el contexto logístico, los datos de entrada, las variables relevantes, los recursos disponibles, las restricciones operativas, los objetivos y las condiciones que hacen válida una solución. **Se utilizarán los datos correspondientes al problema seleccionado por cada equipo**, de acuerdo con los escenarios y materiales proporcionados durante la presentación de la situación problema. Documente sus características y pertinencia.

3. Modelación matemática

Desarrolle una formulación propia del problema elegido que incluya: (a) variables de decisión; (b) parámetros y datos de entrada; (c) función objetivo; (d) restricciones; y (e) representación matemática o computacional de las soluciones. La formulación deberá permitir evaluar cuantitativamente las soluciones. Si se utilizan penalizaciones, justifique su diseño y explique cómo se verifica la factibilidad.

4. Implementación computacional

Utilice como base los códigos proporcionados y desarrollados durante las clases y talleres, adaptándolos al problema seleccionado. El programa en Python deberá:

1. Cargar y procesar los datos del problema y representar las soluciones.
2. Evaluar soluciones mediante una función objetivo o aptitud.
3. Implementar los operadores o mecanismos de búsqueda del algoritmo bioinspirado.
4. Considerar las restricciones y registrar la evolución de las soluciones.
5. Visualizar gráficamente los resultados obtenidos.

Organice el código mediante funciones, con documentación y comentarios pertinentes. Los hiperparámetros principales deberán poder modificarse sin alterar la lógica interna. Explique las adaptaciones realizadas a los códigos de clase y taller.

5. Experimentación y análisis de hiperparámetros

Seleccione **al menos tres hiperparámetros** relevantes, evalúe **al menos tres configuraciones** y realice **10 ejecuciones independientes por configuración** con distintas semillas. Mantenga los datos del problema y condiciones computacionales comparables. Registre y compare, cuando apliquen, las métricas siguientes:

Métrica	Descripción
Mejor solución	Mejor valor de la función objetivo.
Promedio y desviación estándar	Tendencia central y variabilidad entre ejecuciones.
Tiempo de ejecución	Costo computacional requerido.
Factibilidad	Porcentaje de ejecuciones con soluciones válidas.
Convergencia	Evolución de la mejor solución durante las iteraciones.

Incluya gráficas de convergencia y comparaciones entre configuraciones. Analice la influencia de los hiperparámetros en exploración, explotación, convergencia y estabilidad, según corresponda.

6. Evaluación de la calidad de las soluciones

Identifique la mejor solución obtenida, verifique las restricciones, analice las curvas de convergencia y compare las configuraciones experimentales. Explique ventajas, limitaciones y dificultades del algoritmo para el problema seleccionado. **No es obligatorio implementar un segundo algoritmo o método de referencia**; la evaluación se realizará a partir de los experimentos del propio algoritmo.

7. Informe técnico

Elabore un informe en PDF que contenga: (1) introducción y problema seleccionado; (2) modelación matemática; (3) algoritmo bioinspirado y adaptaciones de los códigos vistos en clase y talleres; (4) metodología experimental; (5) resultados con tablas, gráficas y métricas; (6) evaluación y comparación entre configuraciones; (7) discusión de resultados y dificultades; y (8) conclusiones y posibles mejoras. Las conclusiones deberán sustentarse en los experimentos.

8. Reflexión individual: colaboración y aprendizajes de la UF

Cada integrante entregará una reflexión personal, crítica y basada en experiencias concretas, que considere:

- **Trabajo colaborativo:** acuerdos, relaciones interpersonales y cumplimiento de compromisos.
- **Resolución de conflictos:** negociación, respeto y disposición al diálogo.
- **Contribución personal:** responsabilidades, decisiones y aportaciones al proyecto.
- **Aprendizajes adquiridos:** conocimientos teóricos y prácticos de algoritmos bioinspirados, optimización y programación.
- **Experiencia en la UF:** valoración de clases, talleres, actividades, retos y aprendizajes significativos.
- **Aplicación futura:** utilidad de lo aprendido en otras materias, proyectos o contextos profesionales.

9. Evaluación presencial y reserva de horarios

La evaluación se realizará **el jueves 22 de octubre de 2026, en el salón de clases**. Cada equipo contará con 15 minutos para mostrar su programa y responder preguntas. **Las preguntas y la calificación oral serán individuales**; todos los integrantes deberán comprender el proyecto completo.

Opción	Horario disponible
1	11:30–11:45
2	11:50–12:05
3	12:10–12:25
4	12:30–12:45
5	12:45–13:00

Reserva: Un integrante del equipo deberá informar **vía correo electrónico al profesor, tan pronto como hayan acordado su horario de preferencia**, indicando el horario solicitado y los nombres de los integrantes. La asignación está sujeta a disponibilidad y requiere confirmación del profesor.

Entregables y especificaciones

- **Código fuente:** archivos `.py` o `.ipynb`, organizados, comentados, documentados y ejecutables.
- **Informe técnico:** PDF grupal con resultados, tablas, gráficas, metodología y conclusiones.
- **Evidencia experimental:** parámetros, semillas y archivos necesarios para reproducir los experimentos.
- **Reflexión personal:** PDF individual sobre el proyecto y la experiencia en la UF.
- **Declaración de IA:** obligatoria, incorporada al informe técnico.
- **Medio de entrega:** Canvas. Duración estimada: 16–20 horas por estudiante.

Declaración sobre uso de herramientas de Inteligencia Artificial

Se permite el uso auxiliar de IA generativa, **sin superar el 30 % del trabajo realizado**. Puede utilizarse para consultar documentación, comprender conceptos, identificar errores y sugerir mejoras. La modelación, decisiones de diseño, adaptación, experimentación, análisis y conclusiones deberán reflejar el razonamiento propio. Se deberán usar como base los códigos de clases y talleres. **Al final del informe PDF incluya obligatoriamente una declaración:**

- **A.** No utilizamos herramientas de IA generativa para realizar esta actividad (0 %).
- **B.** Utilizamos herramientas de IA generativa. Indicar asistentes utilizados, actividades, un ejemplo de prompt, partes utilizadas/modificadas/rechazadas, porcentaje estimado de asistencia y una decisión de diseño o análisis propia que puedan justificar.

La declaración es obligatoria aun con 0 % de uso. El uso declarado no implica por sí mismo penalización; podrán aplicarse penalizaciones por uso indiscriminado, superar el límite, omitir la declaración o delegar el análisis requerido a la IA.

Rúbrica de calificación (100 puntos)

Criterio	Aspectos a evaluar	Puntos
Código fuente (equipo)	Documentación y comentarios (5); estructura y organización (5); funcionamiento y requisitos (10).	**20**
Informe técnico (equipo)	Problema y modelación (10); algoritmo y adaptación (7); experimentación (8); análisis y discusión (7); conclusiones y claridad (3).	**35**
Reflexión individual (individual)	Colaboración y conflictos (3); contribución personal (2); conocimientos de la UF (3); experiencia y aplicación futura (2).	**10**
Evaluación oral (individual)	Modelación matemática (7); algoritmo y código (12); hiperparámetros y experimentos (8); interpretación y justificación (8).	**35**
Total		**100**

Calificación en equipo (55 puntos): el código fuente (20) y el informe técnico (35) se evalúan grupalmente; todos los integrantes del mismo equipo reciben la misma calificación en ambos componentes. **Calificación individual (45 puntos):** la reflexión personal (10) y la evaluación oral presencial (35) se califican por separado para cada estudiante, de acuerdo con su desempeño. La calificación final de cada integrante será la suma de los cuatro componentes. También se valorarán las subcompetencias institucionales **SMA0103** y **SMA0402** con los niveles Destacado, Sólido, Básico e Incipiente, conforme a sus rúbricas.

Consideraciones generales

Se permiten librerías de Python para cálculo, datos y visualización; sin embargo, el algoritmo bioinspirado deberá implementarse o adaptarse explícitamente a partir de los códigos de clase y talleres. No basta invocar una función que resuelva íntegramente el problema. No se exige alcanzar el óptimo global. Se valorará la calidad de las soluciones, la modelación, la reproducibilidad y la capacidad de cada integrante para justificar su trabajo.